

Unsafe Object.assign() / Potential Mass Assignment

Assessment Identifier: **WS-STA-05** | Assessment Type: **Static** | Date: **September 08, 2026**

Severity: HIGH	CVSS: 7.1	CWE: CWE-915
OWASP: A01	WASC ID: N/A	Confidence: High candidate
Target / Component: src-tauri/sidecar/local-api-server.mjs	Affected URL: https://github.com/worldmonitor/worldmonitor/blob/main/src-tauri/sidecar/local-api-server.mjs	Parameter: Request body in Object.assign() merge

1. Description

Static analysis identified Object.assign() involving application-controlled objects. This can become dangerous when client-controlled properties are copied into security-sensitive objects without an allowlist.

2. Steps to Reproduce

- Run the local API server.
- Identify request fields reaching Object.assign().
- Determine whether those fields are client-controlled.
- Submit a harmless unexpected property.
- Check whether it is copied into the target object.

3. Controlled Proof of Concept

Safe Proof of Concept:
Use a harmless property such as {"unexpectedField":"test"} and observe whether it is copied. Do not modify privileged or security-sensitive fields.

4. Business & Security Impact

If users can modify fields they should not control, mass assignment may allow unauthorized changes to application state or security-related properties.

5. Remediation Recommendations

Primary Fix: Use explicit property allowlists or construct objects field-by-field. Do not blindly merge request objects into privileged objects.

- Destructure only expected, safe fields from incoming request payloads.
- Avoid Object.assign() or spread operators directly with unchecked user request bodies.
- Apply schema validation libraries (such as Zod) to reject unexpected or unknown keys.

6. Evidence & Validation

Evidence Type: Textual / Static Source Analysis
This finding was identified through static analysis of repository source code and configuration files. Per assessment requirements, no simulated runtime screenshots or dynamic PoC captures are applicable. Verification must be performed via source code review and static analysis tool verification.

Authorization & Safety Compliance: *This security assessment was conducted strictly under authorized parameters in accordance with ethical hacking standards. All validations and safe proof-of-concept tests were non-destructive. No persistent modifications or sensitive data exfiltration occurred.*